

Integrating Path Signatures and Transformers: A Novel Approach for Calibrating Rough Volatility Models*

Abstract

In this paper, we present a new model, named the SigFormer¹² model, based on both the self-attention mechanism and the signature transform to calibrate volatility processes simulated by various rough volatility models. We also provide numerical examples to demonstrate its advantages in the calibration of both single and multiple parameters. Code is available at github.com/changanluoxue/signature-transform.

³The calibration of stochastic differential equations is crucial for several applications, especially in quantitative finance. This work presents an innovative machine learning method for calibrating rough volatility models, specifically rough Bergomi and rough Heston models. We leverage path signatures for feature mapping and transformers for their attention mechanism to calibrate both the Hurst parameter and other model parameters efficiently. Our empirical study demonstrates the superiority of our approach in terms of accuracy for the calibration for both single and multiple parameters over existing methods, including convolutional neural networks ([stone'2020](#)) and deep signature techniques ([Lyons'2019'DeepSignature](#)). The code is available at github.com/changanluoxue/signature-transform.⁴

1 Introduction

The effective calibration for rough volatility models represents a crucial challenge in the field of mathematical finance. Over the years, various neural networks based calibration approaches have been developed. These approaches

*Chiheb's edits are in red

¹I think we are not exactly using same architecture as the authors of Sigformer, so maybe better to avoid using same acronym

²"ConvSigMA" is one possible alternative name for our model, since it combines elements from Convolutional layers, path Signature transform and Multi-head Attention mechanisms.

³Chiheb's abstract version

⁴For the code link, we may create later another separate repository (from the current internal one) having only the code.

can be categorized by the different types of neural networks they are based. Since the fully connected feed forward neural network is the basic neural network model, there are many references use it directly for calibration (see e.g., **horvath2021deep**, **rosenbaum2021deep**). The fully connected feed forward neural network can also be integrated into the stochastic Volterra integral equations to calibrate more precisely (see e.g., **wang2021NSDE**, **ma2023rough**). Furthermore, **stone'2020** verified that the convolutional neural network can also be used for calibrating rough volatility models. **Lyons'2019'DeepSignature** introduced the signature based neural network to calibrate fractional Brownian motions.

Recently, large-scale natural language processing models (LLMs) have been developed rapidly thanks to the transformer model proposed by **vaswani'2017**. It is nature to think that whether the transformer can improve the performance of existing neural networks based calibration approaches since the volatility data is also sequential. The aim of this paper is to explore a appropriate combination method for transformer and existing neural networks based calibration approaches to calibrate rough volatility models.

In this paper, we use the self-attention mechanism with signature transform as a layer in a neural network to investigate whether this architecture can help to calibrate parameters from a set of rough paths and we follow **tong'2023** to call it SigFormer model. The structure of the paper is the following. In Section 2, we give a brief introduction to the rough volatility models, path signature theory and attention mechanism that we will use in this paper. The specific architecture of our SigFormer model is clarified in Section 3. In Section 4, we begin with a simple SigFormer model, only consisting of the signature transform layer and single-head attention layer, to present the evolution leading to our SigFormer model. Finally, we discuss our conclusions in Section 5.

2 Background

In this section, we first formalize the calibration problem and then give a brief but necessary introduction to the rough volatility models, path signature theory and attention mechanism.

2.1 Problems formalization

In our context, the rough volatility model calibration is meant to find the model parameters such that the model is as close as possible to the observed volatility path. To formalize this, let us first define a stream $X^n \in \mathbb{R}^{n \times d}$ with $n, d \in \mathbb{N}^+$ as the observed path and $\theta_V \in \mathbb{R}^{l_V}$ with $l_V \in \mathbb{N}^+$ represents parameters corresponding to the observed path in the rough volatility model i.e. the target of this calibration problem. Then the rough volatility model calibration can be

formalized as

$$\hat{\theta}_M = \arg \min_{\theta_M} \delta(\mathcal{M}(X^n; \theta_M), \theta_V), \quad (1)$$

$$\hat{\theta}_V = \mathcal{M}(X^n; \hat{\theta}_M), \quad (2)$$

where $\mathcal{M}(\theta_M) : \mathbb{R}^{n \times d} \mapsto \mathbb{R}^{l_V}$ is an abstract model with parameters $\theta_M \in \mathbb{R}^{l_M}$, $l_M \in \mathbb{N}^+$ and $\delta(\cdot, \cdot)$ is a suitable choice of metric.

2.2 Rough volatility processes

We begin this section by introducing the fractional Brownian motion (fBm), before moving on to describe the rough Bergomi (rBergomi) volatility process and rough Heston (rHeston) volatility process.

Fractional Brownian motion. There are many different integral representations of fBm, here we introduce the characterisation provided by **mandelbrot'1968'fbm** as

$$\begin{aligned} W_t^H &= \frac{1}{\Gamma(\frac{1}{2} + H)} \int_{-\infty}^0 ((t-s)^{H-\frac{1}{2}} - (-s)^{H-\frac{1}{2}}) dB_s \\ &+ \frac{1}{\Gamma(\frac{1}{2} + H)} \int_0^t (t-s)^{H-\frac{1}{2}} dB_s, \quad t \in [0, T] \end{aligned} \quad (3)$$

where T is a positive constant, $H \in (0, 1)$ represents the Hurst parameter, B is a standard Brownian motion and Γ is the standard Gamma function. It is hard to simulate the paths of fBm, as (3) contains an integral process and the kernel $(t-s)^{H-\frac{1}{2}}$ with $H \in (0, \frac{1}{2})$ is singular at point $s = t$.

Rough Bergomi volatility process. The rBergomi volatility process is defined as

$$V_t^{rBergomi} = V_0^{rBergomi} \exp \left(\eta \sqrt{2H} Z_t - \frac{\eta^2}{2} t^{2H} \right), \quad t \in [0, T], \quad (4)$$

with

$$Z_t = \int_0^t (t-s)^{H-\frac{1}{2}} dB_s, \quad (5)$$

where $V_0^{rBergomi}$ and η are both strictly positive constant.

Rough Heston volatility process. The rough Heston volatility process is defined as

$$\begin{aligned} V_t^{rHeston} &= V_0^{rHeston} + \frac{1}{\Gamma(\frac{1}{2} + H)} \int_0^t (t-s)^{H-\frac{1}{2}} \kappa_1 (\theta - V_s^{rHeston}) ds \\ &+ \frac{1}{\Gamma(\frac{1}{2} + H)} \int_0^t (t-s)^{H-\frac{1}{2}} \kappa_2 \sqrt{V_s^{rHeston}} dB_s, \quad t \in [0, T], \end{aligned} \quad (6)$$

where $\kappa_1, \kappa_2, \theta$ are all constants.

We note that there is only one parameter H in fBm while there are two and four parameters in rBergomi volatility process (i.e. H, η) and rHeston volatility process (i.e. $H, \kappa_1, \kappa_2, \theta$), respectively. [These parameters are corresponds to the \$\theta_V\$ in Section 2.1.](#)

2.3 Signature

Given a continuous path $X = \{x_t | x_t = (x_t^1, \dots, x_t^d) \in \mathbb{R}^d, t \in [a, b]\}$ with $a, b \in \mathbb{R}$ and $d \in \mathbb{N}^+$, we can define its signature as the collection of iterated integrals

$$\begin{aligned} \text{Sig}(X) &= \left(\int_{a < t_1 < \dots < t_i < b} dx_{t_1} \otimes \dots \otimes dx_{t_i} \right)_{i \geq 1} \\ &= \left(\left(\int_{a < t_1 < \dots < t_i < b} dx_{t_1}^{d_1} \dots dx_{t_i}^{d_i} \right)_{1 \leq d_1, \dots, d_i \leq d} \right)_{i \geq 1}, \end{aligned} \quad (7)$$

where $\otimes$ denotes the tensor product. From Equation (7), we can easily define the i -th signature of path X as

$$\text{Sig}_i(X) = \int_{a < t_1 < \dots < t_i < b} dx_{t_1} \otimes \dots \otimes dx_{t_i}, \quad i = 1, 2, \dots \quad (8)$$

where the "zeroth" term $\text{Sig}_0(X)$, by convention, is equal to 1. With these notations, we can give the definition as follows.

Definition 2.1 (Signature). *The signature of a continuous path $X : [a, b] \mapsto \mathbb{R}^d$ is defined as an infinite series of tensors indexed by the signature order $i \in \mathbb{N}$,*

$$\text{Sig}(X) := (1, \text{Sig}_1(X), \text{Sig}_2(X), \dots, \text{Sig}_i(X), \dots),$$

where $\text{Sig}_i(X)$ for $i = 1, 2, \dots$ are defined as Equation (8).

Furthermore, we can also define the corresponding truncated signature as

$$\begin{aligned} \text{Sig}^N(X) &= \left(\int_{a < t_1 < \dots < t_i < b} dx_{t_1} \otimes \dots \otimes dx_{t_i} \right)_{1 \leq i \leq N} \\ &= (\text{Sig}_1^N(X), \dots, \text{Sig}_N^N(X)), \end{aligned} \quad (9)$$

with the truncation order $N \in \mathbb{N}^+$ and we exclude the constant "zeroth" term here.

Remark 2.1. [As mentioned in Section 2.1, we interpret a stream of data \$X^n = \(x_{t_1}, \dots, x_{t_n}\)^T \in \mathbb{R}^{n \times d}\$ as a discretisation of path \$X\$ on \$n \in \mathbb{N}^+\$ time grids \$a = t_1 < \dots < t_n = b\$. Then the \(truncated\) signature of \$X^n\$ is defined as the \(truncated\) signature of its linear interpolation.](#)

Remark 2.2. The signature $\text{Sig}^N(X^n)$ has

$$\sum_{i=1}^N d^i = \frac{d^{N+1} - d}{d - 1}$$

components. Obviously, the number of components does not depend on the length of data. Thus the signature is an excellent way to tackle long streams of data.

There are some good properties of signature that make it attractive when dealing with streams of data and here we present these without proof.

Proposition 2.1 (Uniqueness (**Lyons’2010’uniqueness**)). *Given the time-augmented path as $\hat{X}^n := ((x_{t_1}, t_1), \dots, (x_{t_n}, t_n))^T \in \mathbb{R}^{n \times (d+1)}$. Then $\text{Sig}(\hat{X}^n)$ can uniquely determines $\hat{X}^n$.*

The Proposition 2.1 means that there is no information lost during the signature transform. The reason why we need the time-augmented path instead of just the path is the signature transform is invariance to time reparameterisations, which means the signature transform only encodes the order in which data arrives without caring precisely when it arrives.

Proposition 2.2 (Factorial decay (**Lyons’1998’decay**)). *There exists a constant $C(X^n)$ depending on path X^n such that*

$$\|\text{Sig}_i(X^n)\| \leq \frac{C(X^n)^i}{i!},$$

where $\|\cdot\|$ is any tensor norm on $(\mathbb{R}^d)^{\otimes i}$.

The Proposition 2.2 indicate that the terms of the signature decay in size factorially. This ensures that the most significant contributions are given by the lower order terms which makes it reasonable to use truncated signature.

2.4 Transformer and attention mechanism

Transformer is first proposed by **vaswani’2017** for natural language processing. The key component in transformer is self-attention mechanism.

Self-attention mechanism. Consider the discretized path $X^n \in \mathbb{R}^{n \times d}$, and let $W_Q \in \mathbb{R}^{d_{\text{attn}} \times d}$, $W_K \in \mathbb{R}^{d_{\text{attn}} \times d}$ and $W_V \in \mathbb{R}^{d_{\text{attn}} \times d}$ with $d_{\text{attn}} \in \mathbb{N}^+$. Then we can define the query, key and value in self-attention mechanism as $Q = X^n W_Q^T$, $K = X^n W_K^T$ and $V = X^n W_V^T$, respectively, which are linear projections from the path. Then the self-attention mechanism function can be defined as

$$\text{Self-Attention}(X^n; W_Q, W_K, W_V) := \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right)V. \quad (10)$$

Intuitively, the self-attention mechanism function can be viewed as the combination of linear and non-linear transformation applied to the input X^n which is similar to the multilayer perceptron (MLP).

Instead of performing a single self-attention mechanism function, it is beneficial to apply the multi-head self-attention mechanism function which performs the self-attention mechanism function in parallel and then linearly projects these concatenated outputs into an appropriate space.

Multi-head attention mechanism. The multi-head self-attention mechanism function can be defined as

$$\begin{aligned} \text{Multi-Head}(X^n; W_{Q_1}, \dots, W_{Q_h}, W_{K_1}, \dots, W_{K_h}, W_{V_1}, \dots, W_{V_h}, W_O) \\ := \text{Concat}(\text{head}_1, \dots, \text{head}_h)W_O, \end{aligned} \quad (11)$$

where W_O denotes the weight of the output projection, $h \in \mathbb{N}^+$ denotes the number of heads and head_i for $i = 1, \dots, h$ are defined by the Equation (10) with $X_i^n \subseteq X^n$, $W_{Q_i}, W_{K_i}, W_{V_i} \in \mathbb{R}^{d_{\text{attn}_i} \times d}$ with $d_{\text{attn}_i} \in \mathbb{N}^+$.

Multi-head self-attention mechanism allows the model to jointly attend to information from different representation subspaces at different positions while averaging inhibits this with single-head self-attention mechanism.

3 Methodology

In this section, we will clarify the specific architecture of our signature transformer (SigFormer) model, and the reasons why we chose this structure will be explained in the numerical experimental section.

3.1 Augmented with convolutional layer

Let's consider a discretized path $X^n = (x_{t_1}, \dots, x_{t_n})^T \in \mathbb{R}^{n \times d}$. According to **Lyons'2019'DeepSignature**, it is more effective to do the feature map before taking the signature. Then, on the top of our SigFormer model, a convolutional layer $\Phi(X^n; W^\Phi, B^\Phi) : \mathbb{R}^{n \times d} \mapsto \mathbb{R}^{\tilde{n} \times \tilde{d}}$ with kernel size $k \in \mathbb{N}^+$ and stride $s \in \mathbb{N}^+$ is applied to perform the feature map

$$\Phi(X^n; W^\Phi, B^\Phi) = W^\Phi \star X^n + \mathbf{1}_{\tilde{n}} B^\Phi, \quad (12)$$

with

$$(W^\Phi \star X^n)_{i,j} = \sum_{i_1=1}^k \sum_{i_2=1}^d W_{j,i_1,i_2}^\Phi X_{s(i-1)+i_1,i_2}^n, \quad i = 1, \dots, \tilde{n}, j = 1, \dots, \tilde{d}, \quad (13)$$

where $\tilde{n} \in \mathbb{N}^+$ and $\tilde{d} \in \mathbb{N}^+$ represent the output dimension, $\mathbf{1}_{\tilde{n}} \in \mathbb{R}^{\tilde{n}}$ is all-ones column vector, $\star$ is the valid cross-correlation operator and $W^\Phi \in \mathbb{R}^{\tilde{d} \times k \times d}$ and $B^\Phi \in \mathbb{R}^{\tilde{d}}$ are weight and bias for the convolutional layer, respectively. Here, we can always find an appropriate $\tilde{n}$ by doing the zero padding.

After this convolutional layer, according to Proposition 2.1, it is beneficial to augment the output with time before taking the signature. And it can also be augmented with the original data X^n when we set $\tilde{n} = n$.

3.2 Taking the signature with lifted data

Taking signature transform will consume the stream-like nature of the data. To explain further, consider a stream of data $Y^{\tilde{n}} = (y_{t_1}, \dots, y_{t_{\tilde{n}}})^T \in \mathbb{R}^{\tilde{n} \times \tilde{d}}$ to be the output of the convolutional layer and already been augmented, here $\tilde{d}$ denotes the number of features and $\tilde{n}$ denotes the length of sequence which indicates there is a relative position for each element in this data. If we take the truncated signature for $Y^{\tilde{n}}$ with truncation order N , then, according to Remark 2.2, the length of output $\text{Sig}^N(Y^{\tilde{n}}) \in \mathbb{R}^{\frac{\tilde{d}^{N+1}-\tilde{d}}{\tilde{d}-1}}$ is not related to $\tilde{n}$ i.e. it is not a sequential data. The solution is to lift the data before taking the signature as follows

$$l(Y^{\tilde{n}}) = (Y_{1:2}^{\tilde{n}}, \dots, Y_{1:i}^{\tilde{n}}, \dots, Y_{1:\tilde{n}}^{\tilde{n}}), \quad (14)$$

where $Y_{1:i}^{\tilde{n}} = (y_{t_1}, \dots, y_{t_i})^T \in \mathbb{R}^{i \times \tilde{d}}$ for $i = 1, \dots, \tilde{n}$. Each element in the lifted data $l(Y^{\tilde{n}})$ is sequential data. Then the output of taking the signature with lifted data is

$$\text{Sig}^N(l(Y^{\tilde{n}})) = (\text{Sig}^N(Y_{1:2}^{\tilde{n}}), \dots, \text{Sig}^N(Y_{1:\tilde{n}}^{\tilde{n}}))^T. \quad (15)$$

Now the output $\text{Sig}^N(l(Y^{\tilde{n}})) \in \mathbb{R}^{(\tilde{n}-1) \times \frac{\tilde{d}^{N+1}-\tilde{d}}{\tilde{d}-1}}$ has another dimension related to $\tilde{n}$ i.e. it is a sequential data with stream-like nature. Furthermore, we chose the stride equals to 1 in Equation (14) but it is also naturally to choose a larger stride to decrease the length of $l(Y^{\tilde{n}})$. There are also some other plausible choices for l such as taking a sliding window.

3.3 Self-attention layer

The signature can be consider as a feature map which helps to extract the information from the input data. So there is still something needed to process the output after taking the signature, and the feed forward neural network (FNN) is a alternate choice as **Lyons'2019'DeepSignature**. Besides FNN, we choose to add another self-attention layer because it will help in multiple parameters calibration and we will verify this in the next numerical experiments.

Consider the output of taking the signature as follows

$$\begin{aligned} \text{Sig}^N(l(Y^{\tilde{n}})) &= (\text{Sig}^N(Y_{1:2}^{\tilde{n}}), \dots, \text{Sig}^N(Y_{1:\tilde{n}}^{\tilde{n}}))^T \\ &= \begin{pmatrix} \text{Sig}_1^N(Y_{1:2}^{\tilde{n}}) & \cdots & \text{Sig}_N^N(Y_{1:2}^{\tilde{n}}) \\ \vdots & \ddots & \vdots \\ \text{Sig}_1^N(Y_{1:\tilde{n}}^{\tilde{n}}) & \cdots & \text{Sig}_N^N(Y_{1:\tilde{n}}^{\tilde{n}}) \end{pmatrix}_{(\tilde{n}-1) \times \frac{\tilde{d}^{N+1}-\tilde{d}}{\tilde{d}-1}}. \end{aligned} \quad (16)$$

We employ a self-attention mechanism function as Equation (10), then the output become Self-Attention($\text{Sig}^N(l(Y^{\tilde{n}}))$), here we omit the weight matrix for simplicity. As we discussed in Section 2.4, it is beneficial to employ a multi-head attention mechanism function instead of a single-head one. Thus, we use a

N -head attention mechanism function and each head corresponds to a different signature level as follows

$$\text{Multi-Head}(\text{Sig}^N(l(Y^{\tilde{n}}))) = \text{Concat}(\text{head}_1, \dots, \text{head}_N) \widetilde{W}_O, \quad (17)$$

where $\text{head}_i = \text{Self-Attention}((\text{Sig}_i^N(Y_{1:2}^{\tilde{n}}), \dots, \text{Sig}_i^N(Y_{1:n}^{\tilde{n}}))^T)$ for $i = 1, \dots, N$. Here we also omit all the weight matrix except the linear projection matrix $\widetilde{W}_O$ for output. After the self-attention layer, we need another feedforward neural network f on the bottom to project the output to a appropriate dimension.

Now we can clarify the specific architecture of our SigFormer model as below and we also present it in Figure 1.

- a convolutional layer with 3 channels and kernel size 3;
- augmented with time and original values;
- lifted as Equation (14) with stride equals to 50;
- taking the signature transform with truncation order $N = 3$;
- the multi-head self-attention layer with the number fo heads $h = N = 3$ and each different head corresponds to a different signature level;
- a fully connected feedforward neural network with 5 hidden layers, each of size 32 and ReLU activation;
- a non-linear transform with sigmoid function.

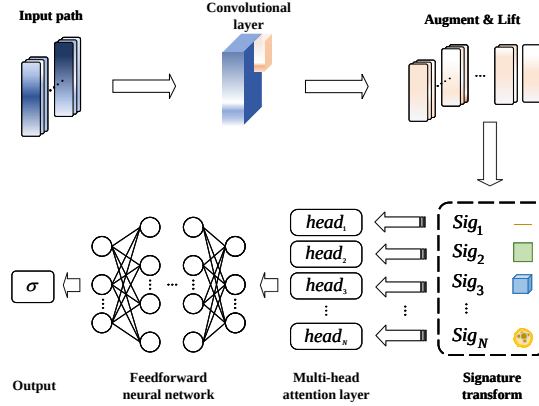


Figure 1: Structure of the SigFormer model.

4 Numerical and empirical experiments

Now we will move on to perform some experiments to verify the advantages of this architecture specified in Section 3 and also to explain why we chose it. To this end, here we choose another three neural network-based models for comparison. The first one is a convolutional neural network (CNN) model proposed by **stone’2020** and the corresponding source code is publicly available on Github⁵. The second one is a signature-based (DeepSigNet) model proposed by **Lyons’2019’DeepSignature** with the code on Github⁶. The third one is a modified transformer model based on **vaswani’2017**. We also present the specific architecture of these three models on Appendix A.1. In the following experiments, we will use these three models as benchmarks and we will show the number of model parameters to represent the model complexity. This section is split into four subsections. We first do some sensitivity analysis to determine the optimal architecture of the SigFormer model. Then we will discuss the single and multiple parameters calibration problem in Section 4.2 and Section 4.3, respectively. Finally, we will provide an empirical study with realized volatility data from financial market in Section 4.4.

4.1 Sensitivity analysis

In this section, we first run sensitivity analysis with respect to the strides and signature truncation order to determine the optimal structure for our SigFormer model. Then we perform examples to verify the effective of CNN and MLP in our SigFormer model.

4.1.1 Implementation details

In order to generate data sets containing paths with different roughness, we first sample five Hurst parameters H from different probability distributions: the Uniform distribution⁷ on $(0, 0.5)$ and the Beta(1, 9) distribution⁸. Obviously, paths with H in the Beta distributed set is rougher than those with H in uniformly distributed set. Then we use these different H to simulate fBm, rBergomi and rHeston paths with different number of time steps $\{100, 200, 300, 400, 500\}$ to generate ten corresponding training/test data sets with the same size as shown in Table 1.

⁵<https://github.com/henrymstone/rough-calibration-using-CNNS>

⁶<https://github.com/patrick-kidger/Deep-Signature-Transforms>

⁷The corresponding set of possible H values is $\{0.06, 0.12, 0.27, 0.35, 0.45\}$.

⁸The corresponding set of possible H values is $\{0.04, 0.08, 0.11, 0.16, 0.21\}$.

Table 1: Training and test data size.

Data set	Number of samples
Training set	3000
Test set	1000

We employ the Cholesky decomposition to simulate fBm and then obtain rBergomi paths with $\eta = 1$ and $V_0^{rBergomi} = 0.01$; this very well-known simulation technique is recommended because the resulting sample paths have the exact distribution, rather than an approximate distribution, of the normalised log volatility process of the rBergomi volatility process. And we apply the fast algorithm proposed by **ma’2022’fast** with $\kappa_1 = 0.1$, $\kappa_2 = 0.03$, $\theta = 0.3$, $V_0^{rHeston} = 0.01$ to simulate rHeston paths.

We use adaptive moment estimation (Adam) optimizer to train these models, setting *batch size* = 64, *epochs* = 100 and *learning rate* = 0.0001 as is fairly standard. We use the mean square error (MSE) as the loss function for training and provide the root mean square error (RMSE) in results as reference.

4.1.2 Sensitivity analysis with respect to strides

As discussed in Section 3.2, the lifting operation before taking the signature could preserve the stream-like nature of the data and then improve the performance of the SigFormer model. However, the lifting operation with smaller stride will lead to more times signature transforms which are time consuming and will also extend the input length of the multi-head self-attention layer and subsequently increase the number of model parameters. Since the lifting operation adds complexity to the SigFormer model, it is important to determine an optimal stride to balance the model complexity and its performance.

In this example, we will use the paths from Section 4.1.1 with the number of time steps equals to 100 for simplicity. We modify the stride of the lifting operation to vary from $\{1, 10, 50, 100\}$ while keeping other settings the same as illustrated in Section 4.1.1. Here we notice that *stride* = *input length* = 100 means there is actually no lifting.

The results are shown in Table 2 with the corresponding loss plots shown in Appendix B.1. From the results, we can observe that the accuracy slightly affected by the choice of stride and there is accuracy lost without the lifting operation in some cases. But the slight improvement in model performance is accompanied by a sharp increase in complexity, it suggests to choose a large stride for a balance.

Table 2: Test results for SigFormer model, averaged over 3 training runs, with the stride varying from 1 to 100. Here the input length equals to 100.

Distribution of H	Stride	Test RMSE			#Params
		fBm	rBergomi	rHeston	
$H \sim \text{Uniform}(0, 0.5)$	1	1.56×10^{-2}	5.23×10^{-2}	8.96×10^{-2}	573306
	10	1.44×10^{-2}	5.28×10^{-2}	9.17×10^{-2}	126906
	50	1.24×10^{-2}	5.43×10^{-2}	8.76×10^{-2}	87226
	100	1.13×10^{-2}	5.47×10^{-2}	7.58×10^{-2}	82266
$H \sim \text{Beta}(1, 9)$	1	1.94×10^{-2}	3.92×10^{-2}	5.73×10^{-2}	573306
	10	1.37×10^{-2}	4.17×10^{-2}	5.81×10^{-2}	126906
	50	1.20×10^{-2}	4.25×10^{-2}	5.87×10^{-2}	87226
	100	1.15×10^{-2}	4.02×10^{-2}	5.83×10^{-2}	82266

4.1.3 Sensitivity analysis with respect to signature truncation order

Following the analysis at the beginning of Section 4.1, we aim to explore the effects of varying signature truncation orders on our SigFormer model next.

In this example, we will use the same paths as Section 4.1.2 but only with $H \sim \text{Beta}(1, 9)$. We modify the signature truncation order vary from $\{1, 2, 3, 4, 5\}$ while keeping other settings the same as Section 4.1.1. Besides the SigFormer model, we will also provide the DeepSigNet model for analysis since there is also signature transform in it. And we propose another simple SigFormer model, whose architecture is presented in Appendix A.2, to validate the necessity of both the convolutional and the fully connected layers in our SigFormer model as well as show our SigFormer is beneficial from multi-head attention. From the previous example, we determine a large stride equals to 50 since we want to take the lifting operation.

The results are shown in Table 3 with the corresponding loss plots shown in Appendix B.2. In order to present the results more clearly, we also plot Table 3 into Figure 4.1.3. It is evident that the optimal signature truncation orders for our SigFormer model vary across different volatility models and more complex volatility models require higher signature truncation order. We also notice that the signature truncation order is correlated with the model complexity, thus we select the truncation order equal to 3 in our SigFormer model for the balance between performance and complexity. We notice that all these three signature based models have the same performance with the signature truncation order equals to 1 since the rough volatility processes are path dependent which could not be specified just by increments. And we can also observe that a higher signature truncation order could not improve the performance of the simple SigFormer model especially for the rBergomi and rHeston volatility processes. We explain this by the single-head attention layer in the simple SigFormer model

could not extract the information well from the path signature comparing with the multi-head attention layer.

Table 3: Test results for different models, averaged over 3 training runs, with the signature truncation order varying from 1 to 5. Here $H \sim \text{Beta}(1, 9)$ and the input lengths equal to 100.

	Test RMSE			#Params	Truncation order
	fBm	rBergomi	rHeston		
DeepSigNet	4.28×10^{-2}	5.99×10^{-2}	6.08×10^{-2}	4461	1
SigFormer	4.18×10^{-2}	5.99×10^{-2}	6.08×10^{-2}	4726	
Simple SigFormer	5.91×10^{-2}	5.99×10^{-2}	6.08×10^{-2}	1086	
DeepSigNet	8.93×10^{-3}	5.60×10^{-2}	6.06×10^{-2}	5261	2
SigFormer	9.16×10^{-3}	5.17×10^{-2}	6.03×10^{-2}	9101	
Simple SigFormer	5.55×10^{-2}	5.95×10^{-2}	6.08×10^{-2}	2946	
DeepSigNet	1.50×10^{-2}	5.37×10^{-2}	6.04×10^{-2}	9261	3
SigFormer	1.14×10^{-2}	4.14×10^{-2}	5.85×10^{-2}	87226	
Simple SigFormer	3.86×10^{-2}	5.95×10^{-2}	6.08×10^{-2}	6666	
DeepSigNet	3.57×10^{-2}	5.30×10^{-2}	6.03×10^{-2}	29261	4
SigFormer	2.37×10^{-2}	4.06×10^{-2}	5.13×10^{-2}	1884101	
Simple SigFormer	3.13×10^{-2}	5.95×10^{-2}	6.08×10^{-2}	14106	
DeepSigNet	2.76×10^{-2}	5.31×10^{-2}	6.04×10^{-2}	129261	5
SigFormer	5.28×10^{-2}	4.06×10^{-2}	4.94×10^{-2}	46024726	
Simple SigFormer	2.77×10^{-2}	5.95×10^{-2}	6.08×10^{-2}	28986	

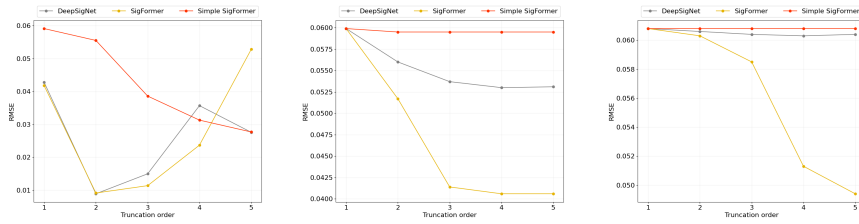


Figure 2: RMSE of calibrating H for different volatility models (Left: fBms; Middle: rBergomi volatility processes; Right: rHeston volatility processes), averaged over 3 training runs, over signature truncation order varying from 1 to 5.

4.1.4 The effective of CNN and MLP in the SigFormer model

In this section, we want to explore the effective of CNN and MLP structure in the SigFormer model. Notice that the Remark 2 from **tong'2023** has mentioned that the transformer in their model can help to extract representations from signatures instead of using CNN. So we want to see whether the CNN and MLP in our SigFormer model matters to achieve higher accuracy.

In order to validate the effective of both the convolutional and the fully connected layers in our SigFormer model, we propose another three simple signature-based transformer for comparison, the architecture of these three simple SigFormer models are presented in Appendix A.2. And we also provide the Simple Sigformer model from Section 4.1.3 for reference. The results are shown in Table 4 for $H \sim \text{Uniform}(0, 0.5)$ and Table 5 for $H \sim \text{Beta}(1, 9)$ with the corresponding loss plots shown in Appendix B.3.

From these results, we can easily find that the CNN and MLP structure in SigFormer model do help to achieve higher accuracy. The Remark 2 from **tong'2023** means that the transformer could be a better alternative for CNN and MLP in signature-based model. But they haven't discuss whether there is beneficial to combine CNN and MLP with transformer in signature-based model. From the results in Section 4.1.4, the CNN and MLP do help to achieve higher accuracy in our model. We put the CNN before taking signature and the transformer after taking signature, so they play different roles in our SigFormer model. The CNN may enhance the ability of signature transform to extract features from paths while the transformer is used to process the output. To realize the role of CNN in our model, we need to dive into the models' structures. We can observe that the dimension of output from signature transform are $[5, 5^2, 5^3]$ and $[2, 2^2, 2^3]$ for SigFormer with and without CNN respectively. This is because we employ a CNN with 3 channels in the former one to increase the dimensionality of the original path with time (i.e. 2 features to 5 features). The MLP in our SigFormer model is used after the transformer. Without the MLP, we can observe some fluctuations during training processes presented in Appendix B.3. The MLP makes our SigFormer model more stable so as to achieve higher accuracy.

Table 4: Test results for different SigFormer models, averaged over 3 training runs, with input lengths varying from 100 to 300 and $H \sim \text{Uniform}(0, 0.5)$.

Models	Input length	Test RMSE			#Params
		fBm	rBergomi	rHeston	
SigFormer without CNN	100	5.36×10^{-2}	1.43×10^{-1}	1.42×10^{-1}	5647
	200	4.76×10^{-2}	1.45×10^{-1}	1.46×10^{-1}	6543
	300	4.68×10^{-2}	1.47×10^{-1}	1.43×10^{-1}	7439
SigFormer without MLP	100	1.18×10^{-1}	1.16×10^{-1}	8.33×10^{-2}	73328
	200	1.45×10^{-1}	1.12×10^{-1}	6.31×10^{-2}	73638
	300	1.01	1.10×10^{-1}	5.84×10^{-2}	73948
SigFormer without CNN and MLP	100	8.80×10^{-2}	1.43×10^{-1}	1.42×10^{-1}	491
	200	7.67×10^{-2}	1.45×10^{-1}	1.45×10^{-1}	519
	300	8.96×10^{-2}	1.47×10^{-1}	1.42×10^{-1}	547
SigFormer	100	1.44×10^{-2}	5.35×10^{-2}	8.43×10^{-2}	87226
	200	1.42×10^{-2}	4.74×10^{-2}	7.10×10^{-2}	97146
	300	1.55×10^{-2}	4.51×10^{-2}	6.29×10^{-2}	107066
Simple SigFormer	100	7.18×10^{-2}	1.43×10^{-1}	1.43×10^{-1}	6666
	200	6.55×10^{-2}	1.45×10^{-1}	1.47×10^{-1}	6666
	300	6.77×10^{-2}	1.47×10^{-1}	1.44×10^{-1}	6666

Table 5: Test results for different SigFormer models, averaged over 3 training runs, with input lengths varying from 100 to 300 and $H \sim \text{Beta}(1, 9)$.

Models	Input length	Test RMSE			#Params
		fBm	rBergomi	rHeston	
SigFormer without CNN	100	2.53×10^{-2}	6.12×10^{-2}	5.94×10^{-2}	5647
	200	2.43×10^{-2}	5.91×10^{-2}	5.96×10^{-2}	6543
	300	2.21×10^{-2}	5.90×10^{-2}	5.83×10^{-2}	7439
SigFormer without MLP	100	1.25×10^{-1}	5.02×10^{-2}	3.98×10^{-2}	73328
	200	6.09×10^{-1}	4.33×10^{-2}	2.35×10^{-2}	73638
	300	1.08	4.49×10^{-2}	2.17×10^{-2}	73948
SigFormer without CNN and MLP	100	4.34×10^{-2}	6.12×10^{-2}	5.91×10^{-2}	491
	200	4.78×10^{-2}	5.91×10^{-2}	5.94×10^{-2}	519
	300	4.66×10^{-2}	5.90×10^{-2}	5.82×10^{-2}	547
SigFormer	100	1.14×10^{-2}	4.18×10^{-2}	5.76×10^{-2}	87226
	200	1.19×10^{-2}	3.64×10^{-2}	5.56×10^{-2}	97146
	300	1.13×10^{-2}	3.24×10^{-2}	5.03×10^{-2}	107066
Simple SigFormer	100	4.07×10^{-2}	6.12×10^{-2}	5.94×10^{-2}	6666
	200	3.53×10^{-2}	5.91×10^{-2}	5.97×10^{-2}	6666
	300	3.46×10^{-2}	5.91×10^{-2}	5.84×10^{-2}	6666

Furthermore, we also want to verify the effective of CNN and MLP in multiple parameters calibration problems. Hence, we sample $\theta \sim \text{Uniform}(0, 0.5)$ ⁹ as well as $H \sim \text{Beta}(1, 9)$ to generate rough Heston volatility paths as Section 4.1.1. The results are shown in Table 6 with the corresponding loss plots shown in Appendix B.3. The analysis of the single calibration problem also applies here.

Table 6: Test results for different SigFormer models, averaged over 3 training runs, with input lengths varying from 100 to 300. Here, $H \sim \text{Beta}(1, 9)$ and $\theta \sim \text{Uniform}(0, 0.5)$.

Models	Input length	Test RMSE	#Params
SigFormer without CNN	100	9.64×10^{-2}	5680
	200	9.78×10^{-2}	6576
	300	9.77×10^{-2}	7472
SigFormer without MLP	100	4.98×10^{-2}	73639
	200	5.11×10^{-2}	74259
	300	4.90×10^{-2}	74879
SigFormer without CNN and MLP	100	7.94×10^{-2}	520
	200	5.94×10^{-2}	576
	300	8.06×10^{-2}	632
SigFormer	100	4.99×10^{-2}	87259
	200	5.10×10^{-2}	97179
	300	5.18×10^{-2}	107099
Simple SigFormer	100	9.94×10^{-2}	6822
	200	1.00×10^{-1}	6822
	300	9.98×10^{-2}	6822

4.2 Calibrating the Hurst parameter from rough volatility paths with different lengths

Estimating the Hurst parameter H of a fBm, rBergomi or rHeston path is considered a nontrivial task because of the paths' non-stationarity and long range dependencies. Therefore paths with more time steps and smaller H (i.e. rougher) are more difficult to calibrate.

⁹The corresponding set of possible θ values is $\{0.05, 0.14, 0.21, 0.30, 0.42\}$.

In this example, we train a variety of models that we mentioned before to perform this estimation. That is, to learn the map $(t^n, X^n) \mapsto H$, where t^n denotes the time steps and X^n are the corresponding simulated paths. We use paths from Section 4.1.1 with the number of time steps varying from 100 to 500 and all other settings are the same as illustrated in Section 4.1.1.

The results are shown in Table 7 for $H \sim \text{Uniform}(0, 0.5)$ and Table 8 for $H \sim \text{Beta}(1, 9)$ with the corresponding loss plots shown in Appendix B.4. In order to present the results more clearly, we also plot Table 7 and Table 8 into Figure 3. From these results, we can easily conclude that the SigFormer model tends to outperform the competing models in three scenarios for single parameter calibration problems: (a) paths with more time steps; (b) rougher paths; (c) paths simulated by more complex volatility models. We attribute this to the self-attention mechanism, which enhances the capacity of the SigFormer model for analyzing information from path signatures, thereby making it outperforming in complex calibration problems. We also observe that the number of parameters i.e. the complexity for the SigFormer model increases with the input length while the DeepSigNet model maintain a constant number of parameters. This feature of the SigFormer model is not directly from introducing the multi-head self-attention layer, but rather through the lifting operation before taking the signature. We also notice that the transformer model has better accuracy than our SigFormer model in some cases. But the transformer model is more complex than our SigFormer model.

Table 7: Test results for different models, averaged over 3 training runs, with input lengths varying from 100 to 500 and $H \sim \text{Uniform}(0, 0.5)$.

Models	Input length	Test RMSE			#Params
		fBm	rBergomi	rHeston	
Transformer	100	8.05×10^{-3}	5.79×10^{-2}	8.82×10^{-2}	573906
	200	6.07×10^{-3}	4.86×10^{-2}	8.00×10^{-2}	1069906
	300	6.78×10^{-3}	5.04×10^{-2}	9.55×10^{-2}	1565906
	400	7.53×10^{-3}	3.76×10^{-2}	8.67×10^{-2}	2061906
	500	1.11×10^{-2}	4.61×10^{-2}	8.17×10^{-2}	2557906
SigFormer	100	1.36×10^{-2}	5.38×10^{-2}	9.06×10^{-2}	87226
	200	1.54×10^{-2}	4.95×10^{-2}	6.99×10^{-2}	97146
	300	1.56×10^{-2}	4.28×10^{-2}	5.49×10^{-2}	107066
	400	1.90×10^{-2}	3.79×10^{-2}	5.47×10^{-2}	116986
	500	1.43×10^{-2}	3.88×10^{-2}	4.36×10^{-2}	126906
DeepSigNet	100	1.69×10^{-2}	6.22×10^{-2}	1.31×10^{-1}	9261
	200	1.69×10^{-2}	5.38×10^{-2}	1.25×10^{-1}	9261
	300	1.62×10^{-2}	4.29×10^{-2}	1.27×10^{-1}	9261
	400	2.09×10^{-2}	4.50×10^{-2}	1.22×10^{-1}	9261
	500	1.81×10^{-2}	4.31×10^{-2}	1.20×10^{-1}	9261
CNN	100	7.24×10^{-2}	5.11×10^{-2}	9.56×10^{-2}	255073
	200	8.42×10^{-2}	4.79×10^{-2}	8.73×10^{-2}	320609
	300	8.66×10^{-2}	5.51×10^{-2}	9.82×10^{-2}	386145
	400	7.76×10^{-2}	4.84×10^{-2}	9.21×10^{-2}	435297
	500	8.51×10^{-2}	4.31×10^{-2}	8.86×10^{-2}	500833

Table 8: Test results for different models, averaged over 3 training runs, with input lengths varying from 100 to 500 and $H \sim \text{Beta}(1, 9)$.

Models	Input length	Test RMSE			#Params
		fBm	rBergomi	rHeston	
Transformer	100	8.97×10^{-3}	5.02×10^{-2}	6.04×10^{-2}	573906
	200	9.43×10^{-3}	4.02×10^{-2}	5.73×10^{-2}	1069906
	300	1.15×10^{-2}	3.25×10^{-2}	5.78×10^{-2}	1565906
	400	7.96×10^{-3}	3.21×10^{-2}	5.82×10^{-2}	2061906
	500	8.05×10^{-3}	2.60×10^{-2}	5.86×10^{-2}	2557906
SigFormer	100	1.28×10^{-2}	3.92×10^{-2}	5.81×10^{-2}	87226
	200	1.78×10^{-2}	3.36×10^{-2}	5.63×10^{-2}	97146
	300	1.93×10^{-2}	2.87×10^{-2}	3.35×10^{-2}	107066
	400	1.48×10^{-2}	3.13×10^{-2}	2.91×10^{-2}	116986
	500	1.75×10^{-2}	2.79×10^{-2}	3.13×10^{-2}	126906
DeepSigNet	100	1.64×10^{-2}	5.18×10^{-2}	6.03×10^{-2}	9261
	200	1.49×10^{-2}	4.42×10^{-2}	5.82×10^{-2}	9261
	300	1.66×10^{-2}	4.23×10^{-2}	5.76×10^{-2}	9261
	400	1.73×10^{-2}	3.70×10^{-2}	5.89×10^{-2}	9261
	500	2.14×10^{-2}	3.56×10^{-2}	5.80×10^{-2}	9261
CNN	100	1.30×10^{-1}	5.69×10^{-2}	8.45×10^{-2}	255073
	200	1.10×10^{-1}	6.15×10^{-2}	8.47×10^{-2}	320609
	300	8.80×10^{-2}	5.59×10^{-2}	8.44×10^{-2}	386145
	400	8.76×10^{-2}	5.77×10^{-2}	7.93×10^{-2}	435297
	500	8.37×10^{-2}	5.84×10^{-2}	8.09×10^{-2}	500833

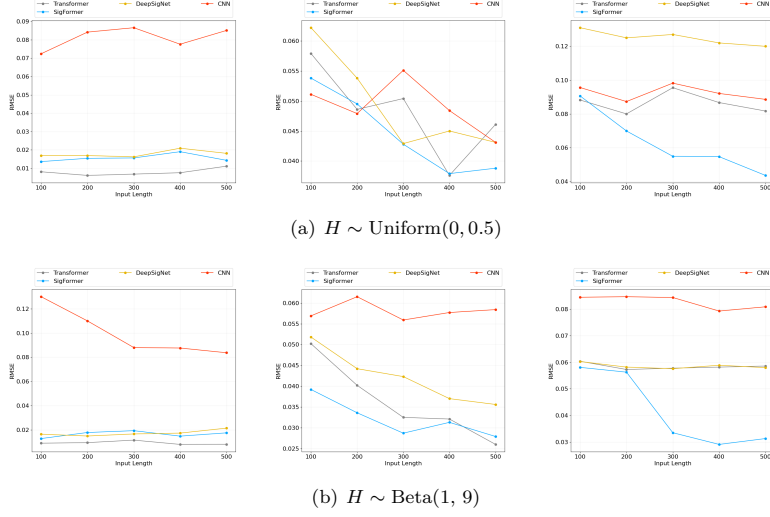


Figure 3: RMSE of calibrating H for different volatility models (Left: fBms; Middle: rBergomi volatility processes; Right: rHeston volatility processes) over input lengths vary from 100 to 500.

4.3 Calibrating multiple parameters with rough volatility processes

In previous examples, we have observed that the SigFormer model tends to outperform competing models in many complex calibration problems, such as those involving paths simulated by advanced volatility models and more time steps. Therefore, it is natural to consider applying the SigFormer model in multiple parameters calibration problems.

4.3.1 Implementation details

In order to generate the data sets for multiple parameters calibration problems, we first sample different $\eta, \kappa_1, \kappa_2, \theta$ from the distribution shown in Table 9. Then we simulate rBergomi and rHeston paths with these parameters, as well as $H \sim \text{Beta}(1, 9)$, from Section 4.1.1 to generate two corresponding training/test data sets with the same size as shown in Table 1 and all paths are simulated 100 steps. The first data set consists of rBergomi paths with different η and H . The second data set consists of rHeston paths with different $\kappa_1, \kappa_2, \theta$ and H . All other settings are kept the same as Section 4.1.1.

Table 9: Distributions for sampling parameters.

Parameters	Distributions	Values
η	Uniform(0, 3)	{0.41, 1.04, 1.56, 2.40, 2.73}
κ_1	Uniform(0, 5)	{0.25, 1.24, 2.83, 3.02, 4.81}
κ_2	Uniform(0, 3)	{0.13, 0.79, 1.58, 2.23, 2.94}
θ	Uniform(0, 0.5)	{0.05, 0.14, 0.21, 0.30, 0.42}

4.3.2 Multiple parameters calibration

We begin this section by calibrating two parameters simultaneously: H , η of rBergomi volatility processes and we also perform four parameters calibrations for rHeston volatility processes (i.e. H , κ_1 , κ_2 and θ). The results are shown in Table 10 with the corresponding loss plots shown in Appendix B.5.

Table 10: Test results for multiple parameters calibration with $H \sim \text{Beta}(1, 9)$, $\eta \sim \text{Uniform}(0, 3)$, $\kappa_1 \sim \text{Uniform}(0, 5)$, $\kappa_2 \sim \text{Uniform}(0, 3)$ and $\theta \sim \text{Uniform}(0, 0.5)$.

Models	RMSE of rBergomi	Params	RMSE of rHeston	Params
Transformer	0.448	573939	0.689	574005
SigFormer	0.289	87259	0.672	87325
DeepSigNet	0.564	9294	0.769	9360
CNN	0.388	255202	0.768	255460

In these multiple calibration problems, we observe that our SigFormer model achieves the best accuracy. And we also notice that models with transformer achieve better accuracy than those without. These results further demonstrate the advantages of combining self-attention mechanism with the signature transform for calibrating complex rough paths.

4.4 Empirical studies

In this example, we follow **stone’2020** to calibrate the Hölder exponent of historic realised volatility data from the Oxford–Man Institute of Quantitative Finance. We generate the training and evaluation data sets by the rough Heston model with two different Hurst parameters i.e. $H \sim \text{Beta}(1, 9)$ and $H \sim \text{Uniform}(0, 0.5)$ as illustrated in Section 4.1.1. We choose the length of the input vectors to be 100. We took a sample of 10 different indices; for each index we then used a time series of 200 sequential data points to create 11 vectors of

length 100 (entries 0 to 100, 10 to 110 and so on) to predict the Hölder exponent for each index. We compute the RMSE between the models’ predictions and the least square prediction, and the standard deviation of the difference between these predictions, see Table 11. We also present the loss plots for training in Appendix B.6. We notice that models trained with $H \sim \text{Beta}(1, 9)$ achieve better accuracy than those who trained with $H \sim \text{Uniform}(0, 0.5)$. This means that the log-volatility behaves essentially as a fBm with Hurst exponent H of order 0.1, at any reasonable time scale.

Table 11: Calibration results for different models with $H \sim \text{Beta}(1, 9)$ and $H \sim \text{Uniform}(0, 0.5)$.

Models	RMSE (Standard deviation)		#Params
	$H \sim \text{Uniform}(0, 0.5)$	$H \sim \text{Beta}(1, 9)$	
SigFormer	0.285(9.30×10^{-3})	0.016(6.09×10^{-4})	87226
DeepSigNet	0.152(5.16×10^{-3})	0.032(5.23×10^{-4})	9261
CNN	0.776(2.46×10^{-2})	0.045(1.75×10^{-3})	255073
Simple SigFormer	0.095(3.34×10^{-3})	0.038(7.11×10^{-4})	6822
Transformer	0.221(7.36×10^{-3})	0.026(3.50×10^{-4})	573906

5 Conclusion

There is a solid theoretical ground for using the signature transform as a tool to calibrate the streams of data. Meanwhile transformers have enjoyed great empirical success in handling sequential data. It is thus natural to combine them together to calibrate volatility processes; in this paper we have constructed a SigFormer model to present how to do this, and also have provided examples to explore the performance of the SigFormer model in different calibration problems.

There are two main contributions in this paper. First, we give the way to combine the signature transform with the self-attention mechanism. Second, we present how to specify the details of the SigFormer model as well as how to simplify it without substantially compromising accuracy.

A Architecture of different models

A.1 Architecture of the CNN, DeepSigNet and transformer model

Here we clarify the specific architecture of CNN, DeepSigNet and transformer model as below:

- The architecture of CNN model:
 - a convolutional layer with 32 channels and kernel size 20;
 - the leaky ReLu transform with negative slope 0.1;
 - a max pooling layer with size 3;
 - a dropout layer with rate 0.25;
 - a convolutional layer with 64 channels and kernel size 20;
 - the leaky ReLu transform with negative slope 0.3;
 - a max pooling layer with size 3;
 - a dropout layer with rate 0.25;
 - a convolutional layer with 128 channels and kernel size 20;
 - the leaky ReLu transform with negative slope 0.1;
 - a max pooling layer with size 3;
 - a dropout layer with rate 0.4;
 - a dense layer with 128 units;
 - the leaky ReLu transform with negative slope 0.1;
 - a dropout layer with rate 0.3;
 - a dense layer with 1 units;
 - a non-linear transform with sigmoid function.
- The architecture of DeepSigNet model:
 - a convolutional layer with 3 channels and kernel size 3;
 - augmented with time and original values;
 - taking the signature transform with truncation order $N = 3$;
 - a fully connected feedforward neural network with 5 hidden layers, each of size 32 and ReLU activation;
 - a non-linear transform with sigmoid function.
- The architecture of transformer model:
 - a convolutional layer with 3 channels and kernel size 3;
 - augmented with time and original values;

- a multi-head self-attention layer with the number of heads $h = N = 3$;
- a fully connected feedforward neural network with 5 hidden layers, each of size 32 and ReLU activation;
- a non-linear transform with sigmoid function.

A.2 Architecture of simple signature-based transformer models

Here we clarify the specific architecture of different SigFormer models as below:

- The architecture of simple SigFormer model:
 - augmented with time;
 - taking the signature transform with truncation order $N = 3$;
 - a single-head self-attention layer;
 - a non-linear transform with sigmoid function.
- The architecture of SigFormer without CNN:
 - augmented with time;
 - taking the signature transform with truncation order $N = 3$;
 - the multi-head self-attention layer with the number of heads $h = N = 3$ and each different head corresponds to a different signature level;
 - a fully connected feedforward neural network with 5 hidden layers, each of size 32 and ReLU activation;
 - a non-linear transform with sigmoid function.
- The architecture of SigFormer without MLP:
 - a convolutional layer with 3 channels and kernel size 3;
 - augmented with time and original values;
 - taking the signature transform with truncation order $N = 3$;
 - the multi-head self-attention layer with the number of heads $h = N = 3$ and each different head corresponds to a different signature level;
 - a linear transform to appropriate dimension.
- The architecture of SigFormer without CNN and MLP:
 - augmented with time;
 - taking the signature transform with truncation order $N = 3$;
 - the multi-head self-attention layer with the number of heads $h = N = 3$ and each different head corresponds to a different signature level;
 - a linear transform to appropriate dimension.

B Loss plots

Here we present the loss plots for a particular (typical) training run since we have 3 training runs for each experiment. And the data in loss plots has been taken the rolling mean with a window size of 5.

B.1 Loss plots for the SigFormer model with the stride varying from 1 to 100

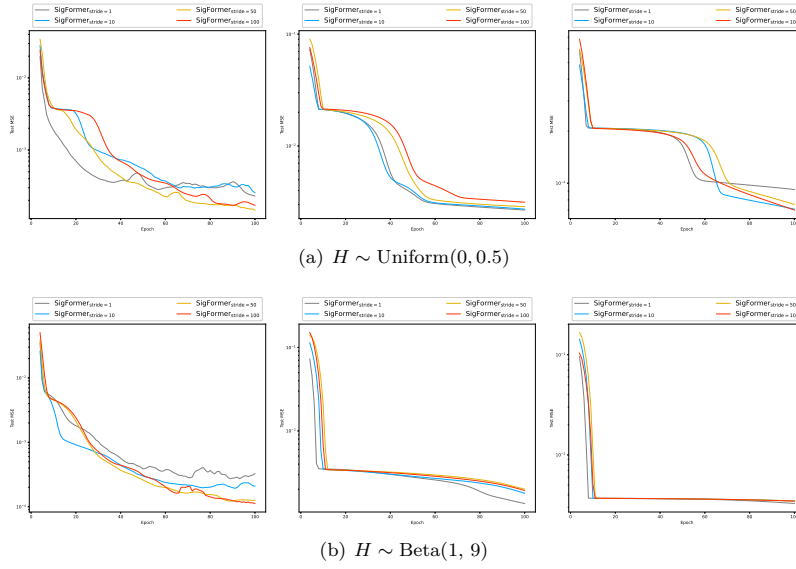


Figure 4: Loss plot for (Left:) fBMs; (Middle:) rBergomi volatility processes; (Right:) rHeston volatility processes with the stride varying from 1 to 100. Here, the input length equals to 100.

B.2 Loss plots for signature-based models with the signature truncation order varying from 1 to 5

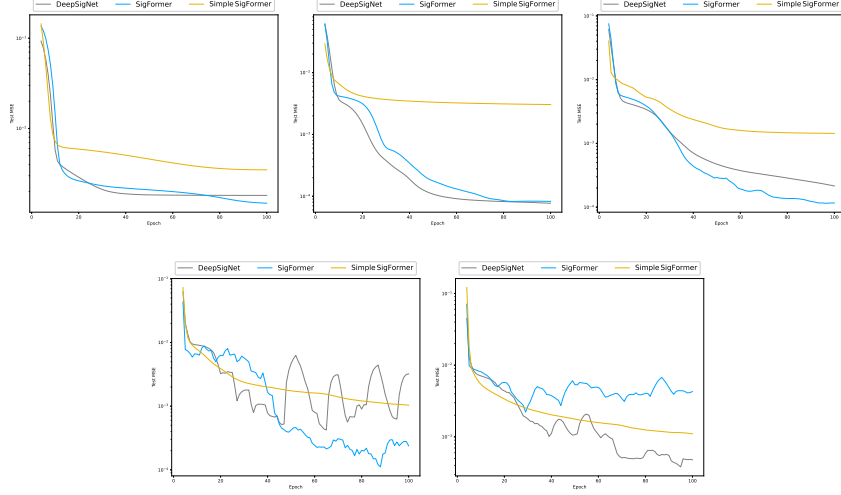


Figure 5: Loss plot for fBms with signature truncation order ranging from (left to right and top to bottom:) $\{1, 2, 3, 4, 5\}$. Here $H \sim \text{Beta}(1, 9)$ and the input lengths equal to 100.

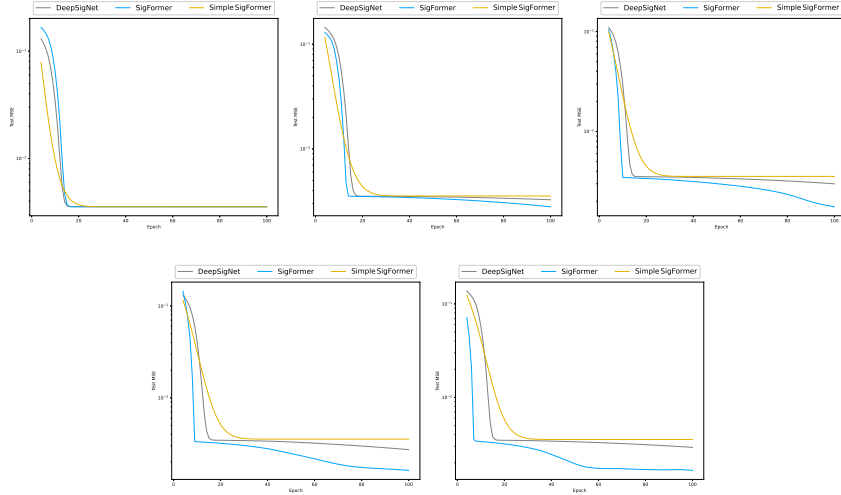


Figure 6: Loss plot for rBergomi volatility processes with signature truncation order ranging from (left to right and top to bottom:) $\{1, 2, 3, 4, 5\}$. Here $H \sim \text{Beta}(1, 9)$ and the input lengths equal to 100.

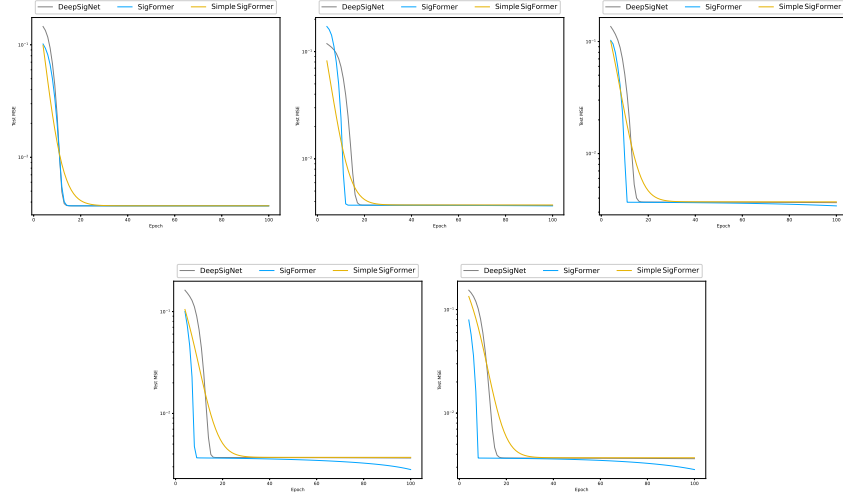
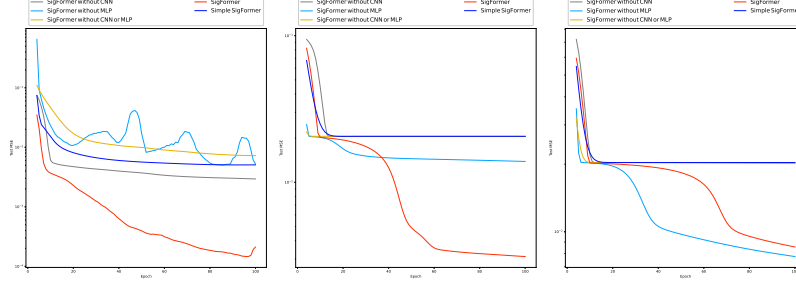
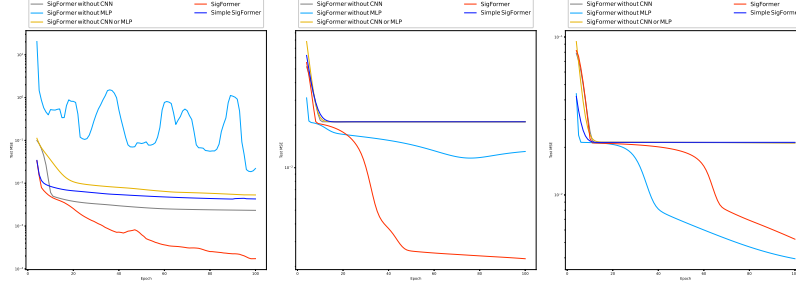


Figure 7: Loss plot for rHeston volatility processes with signature truncation order ranging from (left to right and top to bottom:) $\{1, 2, 3, 4, 5\}$. Here $H \sim \text{Beta}(1, 9)$ and the input lengths equal to 100.

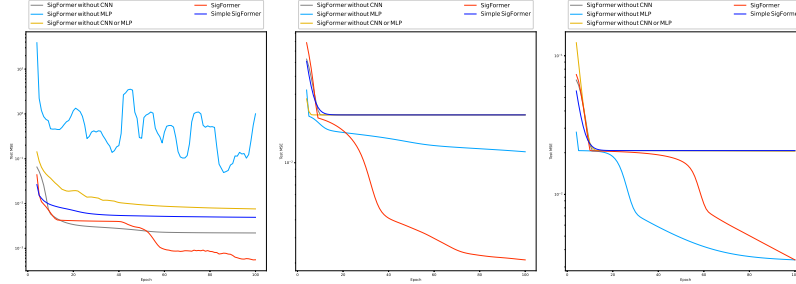
B.3 Loss plots for different SigFormer models with input lengths varying from 100 to 300



(a) The input length equals to 100.

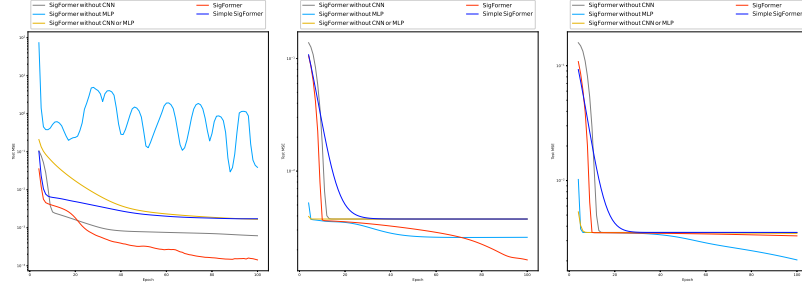


(b) The input length equals to 200.

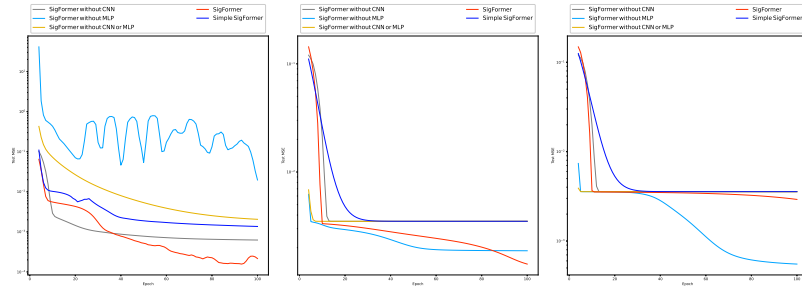


(c) The input length equals to 300.

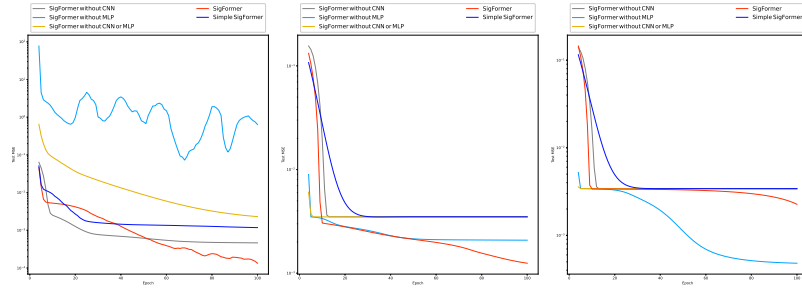
Figure 8: Loss plot for (Left:) fBMs; (Middle:) rBergomi volatility processes; (Right:) rHeston volatility processes with input lengths varying from 100 to 300 and $H \sim \text{Uniform}(0, 0.5)$.



(a) The input length equals to 100.



(b) The input length equals to 200.



(c) The input length equals to 300.

Figure 9: Loss plot for (Left:) fBMs; (Middle:) rBergomi volatility processes; (Right:) rHeston volatility processes with input lengths varying from 100 to 300 and $H \sim \text{Beta}(1, 9)$.

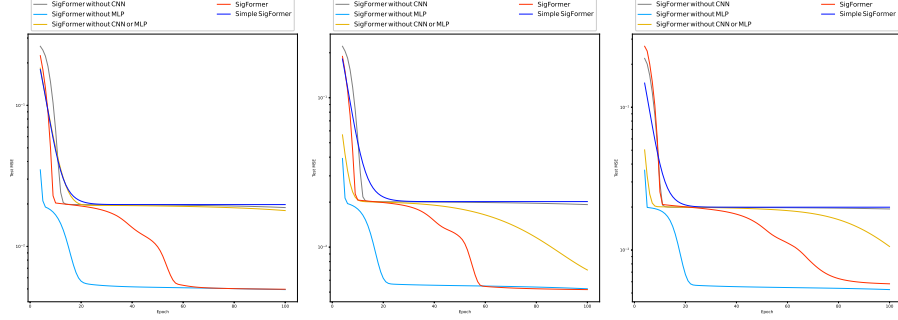


Figure 10: Loss plot for rHeston volatility processes with input lengths varying from 100 to 300 (from left to right). Here, $H \sim \text{Beta}(1, 9)$ and $\theta \sim \text{Uniform}(0, 0.5)$.

B.4 Loss plots for different models with input lengths varying from 100 to 500

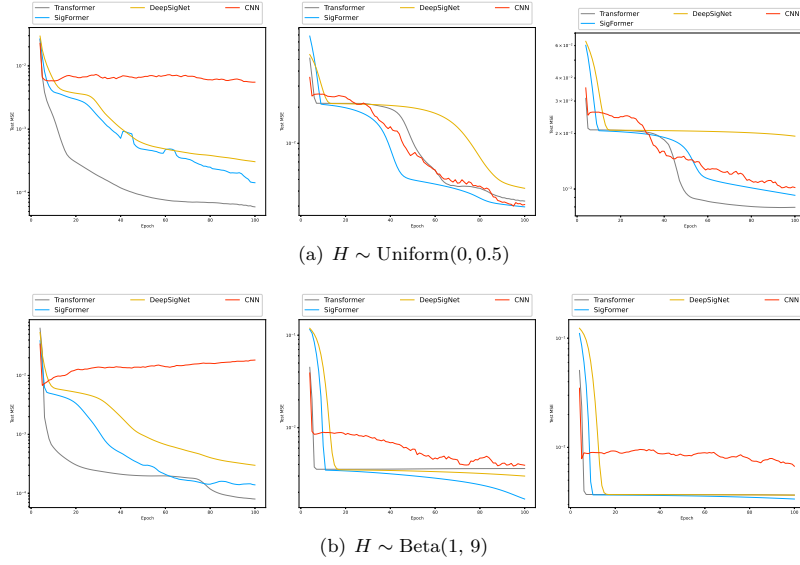


Figure 11: Loss plot for (Left:) fBms; (Middle:) rBergomi volatility processes; (Right:) rHeston volatility processes with input lengths equal to 100.

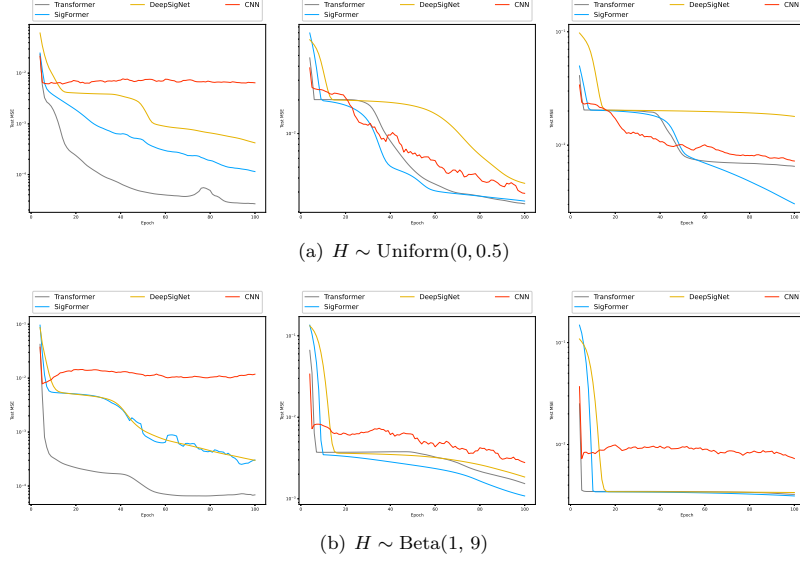


Figure 12: Loss plot for (Left:) fBMs; (Middle:) rBergomi volatility processes; (Right:) rHeston volatility processes with input lengths equal to 200.

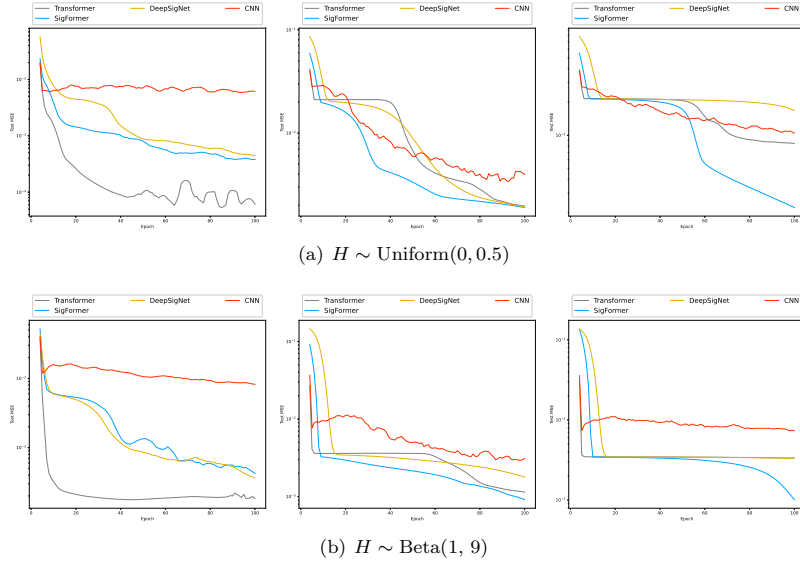


Figure 13: Loss plot for (Left:) fBMs; (Middle:) rBergomi volatility processes; (Right:) rHeston volatility processes with input lengths equal to 300.

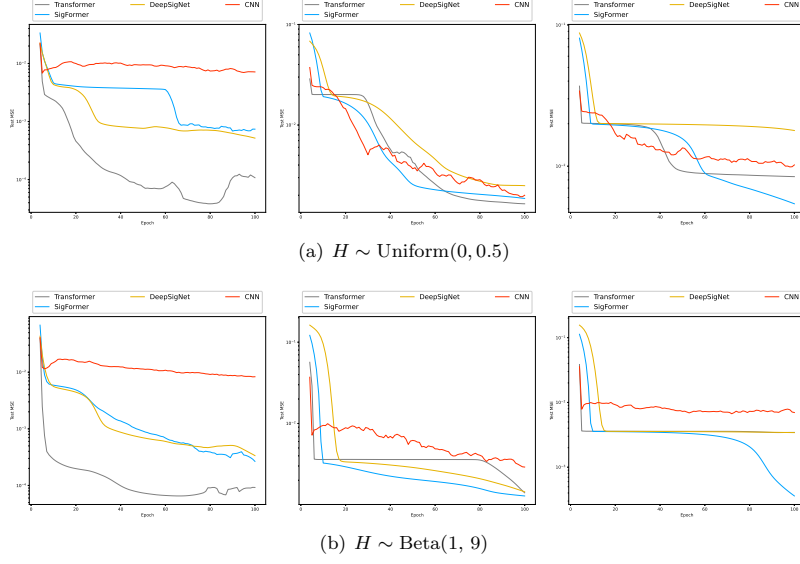


Figure 14: Loss plot for (Left:) fBms; (Middle:) rBergomi volatility processes; (Right:) rHeston volatility processes with input lengths equal to 400.

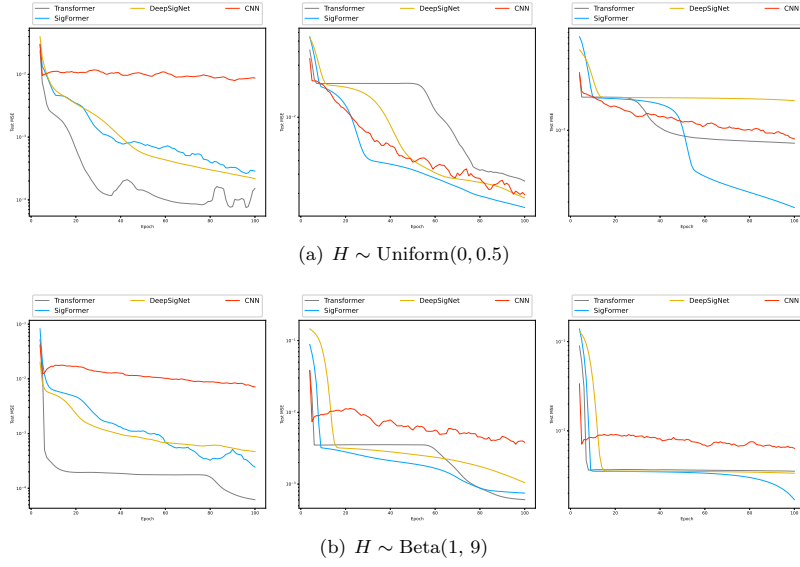


Figure 15: Loss plot for (Left:) fBms; (Middle:) rBergomi volatility processes; (Right:) rHeston volatility processes with input lengths equal to 500.

B.5 Loss plots for multiple parameters calibrations

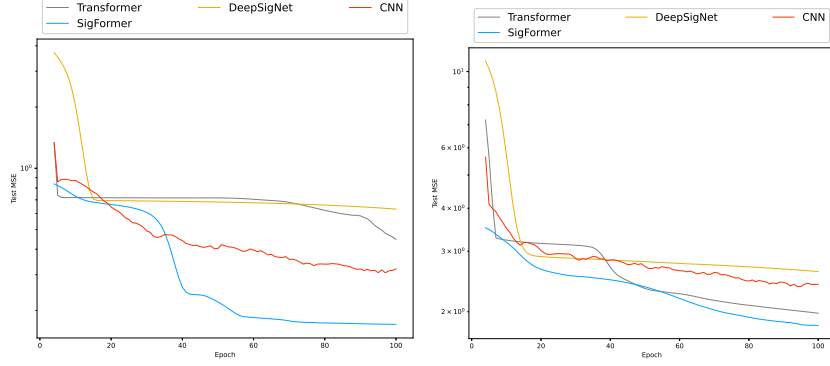


Figure 16: Loss plot of multiple parameters calibration for (Left:) rBergomi volatility processes and (Right:) rHeston volatility processes with $H \sim \text{Beta}(1, 9)$, $\eta \sim \text{Uniform}(0, 3)$, $\kappa_1 \sim \text{Uniform}(0, 5)$, $\kappa_2 \sim \text{Uniform}(0, 3)$ and $\theta \sim \text{Uniform}(0, 0.5)$.

B.6 Loss plots for empirical studies

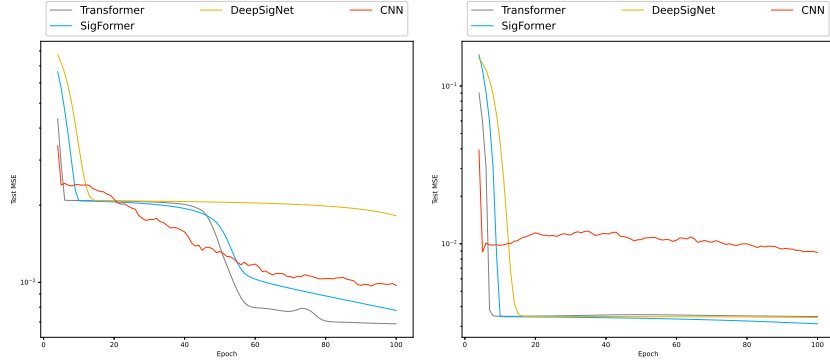


Figure 17: Loss plot of rHeston volatility processes with (Left:) $H \sim \text{Uniform}(0, 0.5)$ and (Right:) $H \sim \text{Beta}(1, 9)$.